

Project
Data Hiding Techniques
&
Recovering the Passwords Techniques

Data-Hiding Techniques:

Data hiding is the process of altering or modifying a file to hide information. Bit-shifting, utilizing encryption, setting up password protection, modifying file properties, hiding entire partitions, changing file extensions, and other methods are examples of data-hiding strategies.

1. slack space:

When examining any storage medium for forensic purposes, slack space is essential. It's can be used to hide data from the operating system and other users. It is a storage area produced when a user-created file does not use the entirety of the space allotted by a file system. It is a form of internal fragmentation, to put it another way. unoccupied space in a disk cluster's allotted area. A cluster is nothing more than the fixed-size storage units that the file system uses. It consists of a number of adjacent sectors. Every file will be kept in the cluster or clusters assigned to it from the very beginning. The file system assigns a certain amount of clusters to each newly produced file. The file size can be greater than, less than, or equal to the sum of the cluster sizes divided by the number of clusters. Rarely does the size of a file match the size of numerous clusters in most circumstances. As a result, there will be a small empty space created between the end of the file's contents and the end of the last cluster the file system assigned to it. File slack, often known as slack space of a file, is this assigned but underutilized area. In other words, file slack is also formed as a result of the logical and physical size discrepancies. The host or carrier file is unaffected while the hidden data is transparent to the host operating system and file managers, a primitive but useful feature of data hiding in file slack. The drawback is that a simple disk editor can quickly recover the hidden message.

2. STEGANOGRAPHY:

Steganography conceals the confidential information in another file so that only the intended recipient is aware it exists. Data was once concealed by being placed on the scalps of slaves, the backs of writing tablets, the stomachs of rabbits, or the backs of wax. However, the majority of individuals in today's world use the medium to send data in the form of text, photos, video, and audio. Confidential data is transmitted safely by using multimedia objects like audio, video, and images as cover sources to hide the data.

Types of Steganography:

Text Steganography: Information is concealed inside text files in this method. This technique conceals the confidential information behind every nth letter of every text message's words. There are many options for data concealing in text files. Format-based methods, random and statistical methods, and linguistics methods are among them.

Image Steganography: Image steganography is the practice of concealing data by representing a cover item as an image. In image steganography, the data is concealed using pixel intensities. Images are a common cover source in digital steganography since a digital representation of a picture contains a number of bits.

Audio Steganography: Data is concealed in audio files. With this technique, data in WAV, AU, and MP3 sound files is concealed. Different techniques exist for audio steganography. Low bit encoding, phase coding, and spread spectrum are some of these techniques.

Video Steganography: This method allows you to conceal any kind of files or data inside of a digital video file. In this instance, the data is concealed via video (a combination of images). Typically, discrete cosine transformations (DCT) change the values (for example, from 8.667 to

9) to disguise the data in each of the video's images in a way that is invisible to the human eye. The file types used for video steganography are H.264, Mp4, MPEG, and AVI.

3. Host protected area:

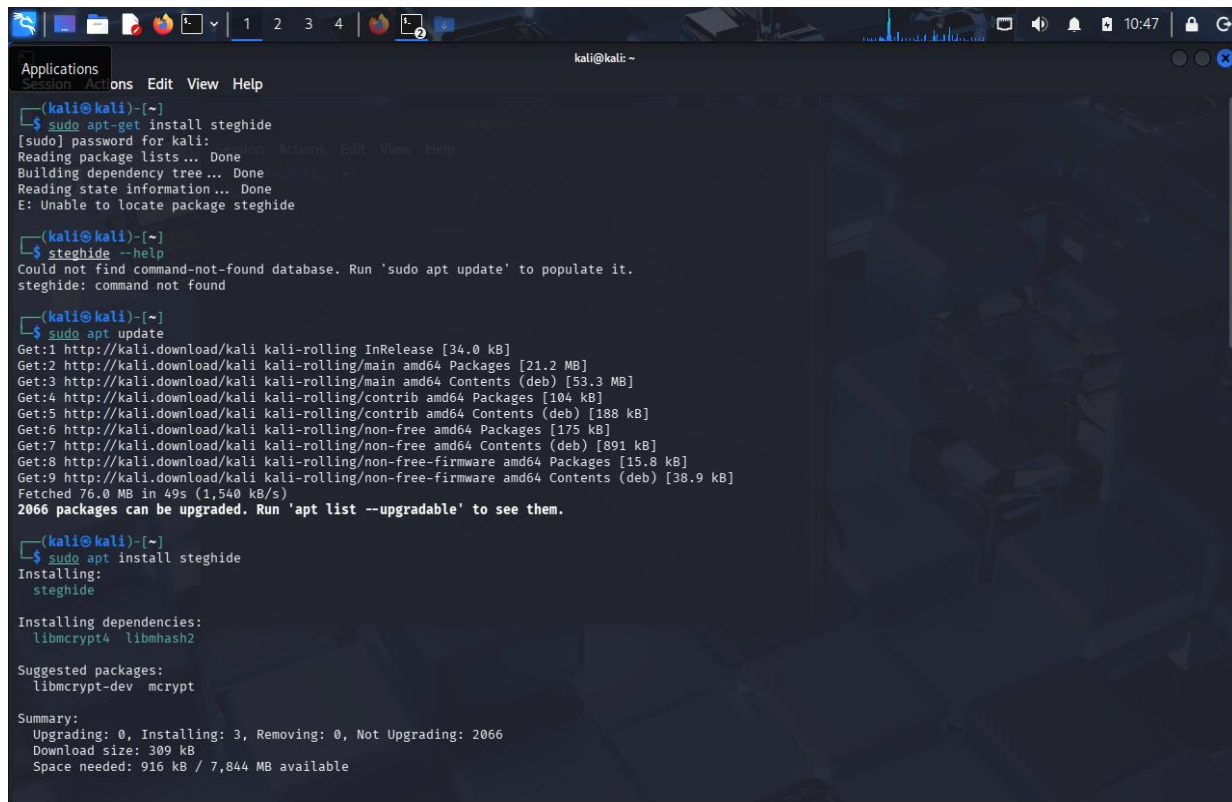
The HPA, or Hidden Protected Space, is a unique area (often a few gigabytes in size) found near the end of a hard disk. It is also referred to as the Host Protected Area and the Predesktop Area. Given that we must determine how much hard drive space should be used by HPA. In order to prevent users from unintentionally erasing the host protected area (HPA), which holds critical information including boot information, it is designed to be concealed. The HPA can be read by some command lines and specialized software, and these tools are typically used to remove data or reduce the size of the HPA. The BIOS can use the HPA to help boot the computer and for diagnostic purposes if the user has a basic input/output system (BIOS) that is HPA-aware. On the HPA, some computer manufacturers might also have an OS pre-installed. When a computer is brought in for repair after being hacked, the technician often uses the HPA to make the necessary adjustments.

4. NTFS Data Streams:

Alternative data streams offer a means of including secret information, like file summary data, with a file. The NTFS file system has a feature known as alternate data streams. They were initially included in Windows NT 3.1 to enable file sharing between NT and Mac users. One or more additional data streams may also be present along with the \$DATA default data stream, which contains the contents of an NTFS file. The \$DATA data stream can be opened, read from, and written to by anyone with the appropriate rights to the file. A user or application can add extra data or files to an NTFS file in the form of an auxiliary data stream. The alternate data stream's existence will only be known to the individual who generated it. Users typically won't be able to tell whether

an additional data stream has been added to a file because the stream's name and content are hidden, and there won't be a noticeable change in the file size. There are various applications for other data streams. To keep summary information for non-Microsoft Office documents like plain text (.txt) files, Windows employs other data streams. On the Summary tab of a file's Properties dialog box, you can enter file summary information such the title, subject, and author. A different data stream called Summary Information holds the summary information.

❖ Implementation:



```
(kali@kali)-[~]
└─$ sudo apt-get install steghide
[sudo] password for kali:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
E: Unable to locate package steghide

(kali@kali)-[~]
└─$ steghide --help
Could not find command-not-found database. Run 'sudo apt update' to populate it.
steghide: command not found

(kali@kali)-[~]
└─$ sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.2 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.3 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [104 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [188 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [175 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [891 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [15.8 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [38.9 kB]
Fetched 76.0 MB in 49s (1,540 kB/s)
2066 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@kali)-[~]
└─$ sudo apt install steghide
Installing:
  steghide

Installing dependencies:
  libmcrypt4 libmhash2

Suggested packages:
  libmcrypt-dev mcrypt

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 2066
  Download size: 309 kB
  Space needed: 916 kB / 7,844 MB available
```

```
kali@kali: ~  
Session Actions Edit View Help  
Processing triggers for kali-menu (2025.4.3) ...  
  
(kali@kali)-[~]  
$ steghide --help  
steghide version 0.5.1  
  
the first argument must be one of the following:  
embed, --embed          embed data  
extract, --extract       extract data  
info, --info             display information about a cover- or stego-file  
info <filename>         display information about <filename>  
encinfo, --encinfo       display a list of supported encryption algorithms  
version, --version        display version information  
license, --license        display steghide's license  
help, --help             display this usage information  
  
embedding options:  
-ef, --embedfile          select file to be embedded  
-ef <filename>            embed the file <filename>  
-cf, --coverfile          select cover-file  
-cf <filename>            embed into the file <filename>  
-p, --passphrase          specify passphrase  
-p <passphrase>          use <passphrase> to embed data  
-sf, --stegofile          select stego file  
-sf <filename>            write result to <filename> instead of cover-file  
-e, --encryption          select encryption parameters  
-e <a>[<m>][<m>[<a>]      specify an encryption algorithm and/or mode  
-e none                   do not encrypt data before embedding  
-z, --compress            compress data before embedding (default)  
-z <l>                    using level <l> (1 best speed... 9 best compression)  
-Z, --dontcompress        do not compress data before embedding  
-K, --nochecksum          do not embed crc32 checksum of embedded data  
-N, --dontembedname       do not embed the name of the original file  
-f, --force               overwrite existing files  
-q, --quiet               suppress information messages  
-v, --verbose             display detailed information  
  
extracting options:  
-sf, --stegofile          select stego file  
-sf <filename>            extract data from <filename>  
-p, --passphrase          specify passphrase  
-p <passphrase>          use <passphrase> to extract data  
-xf, --extractfile        select file name for extracted data
```

```
kali@kali: ~/Desktop  
Session Actions Edit View Help  
  
extracting options:  
-sf, --stegofile          select stego file  
-sf <filename>            extract data from <filename>  
-p, --passphrase          specify passphrase  
-p <passphrase>          use <passphrase> to extract data  
-xf, --extractfile        select file name for extracted data  
-xf <filename>            write the extracted data to <filename>  
-f, --force               overwrite existing files  
-q, --quiet               suppress information messages  
-v, --verbose             display detailed information  
  
options for the info command:  
-p, --passphrase          specify passphrase  
-p <passphrase>          use <passphrase> to get info about embedded data  
  
To embed emb.txt in cvr.jpg: steghide embed -cf cvr.jpg -ef emb.txt  
To extract embedded data from stg.jpg: steghide extract -sf stg.jpg  
  
(kali@kali)-[~]  
$ ls  
Desktop Documents Downloads Music Pictures Public Templates Videos  
  
(kali@kali)-[~]  
$ man steghide  
  
(kali@kali)-[~]  
$ ls  
Desktop Documents Downloads Music Pictures Public Templates Videos  
  
(kali@kali)-[~]  
$ cd Desktop  
  
(kali@kali)-[~/Desktop]  
$ ls  
S1 S1.jpeg  
  
(kali@kali)-[~/Desktop]  
$ cat S1  
this is secret message  
  
(kali@kali)-[~/Desktop]  
$
```



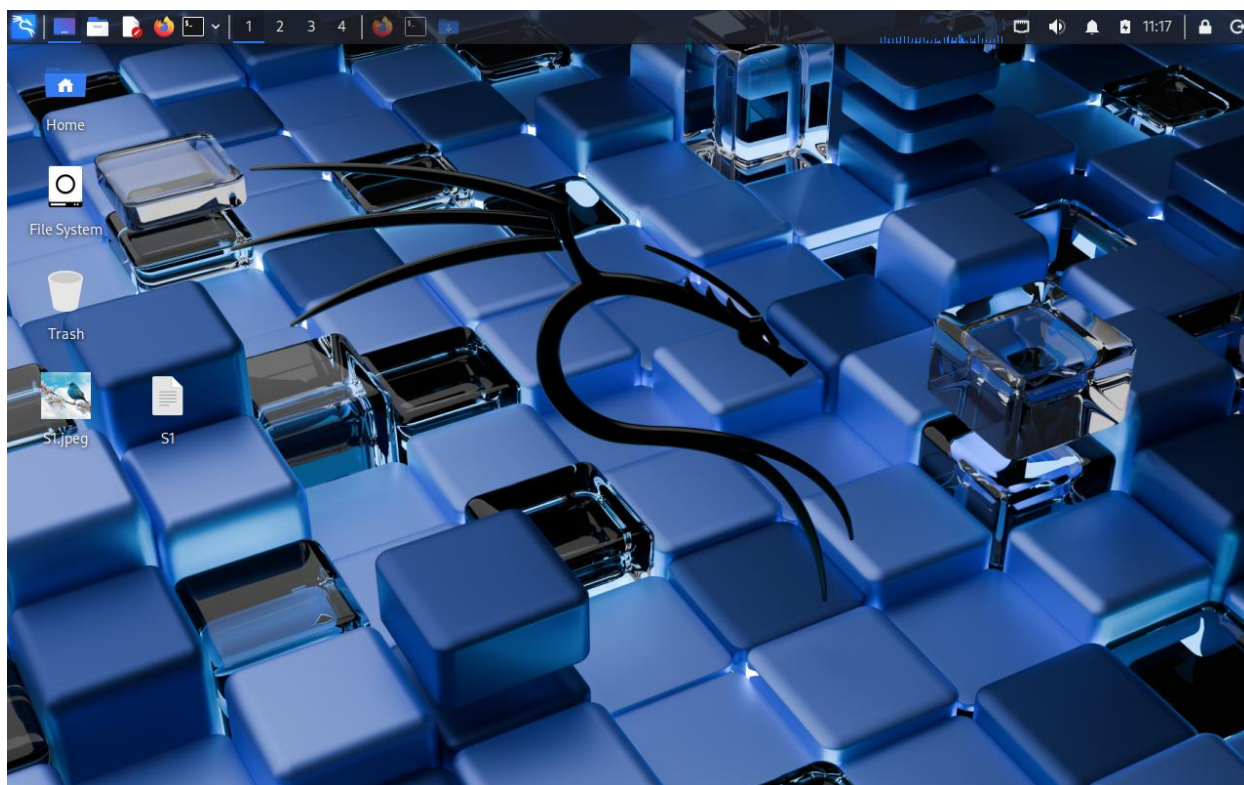
```
kali@kali: ~/Desktop

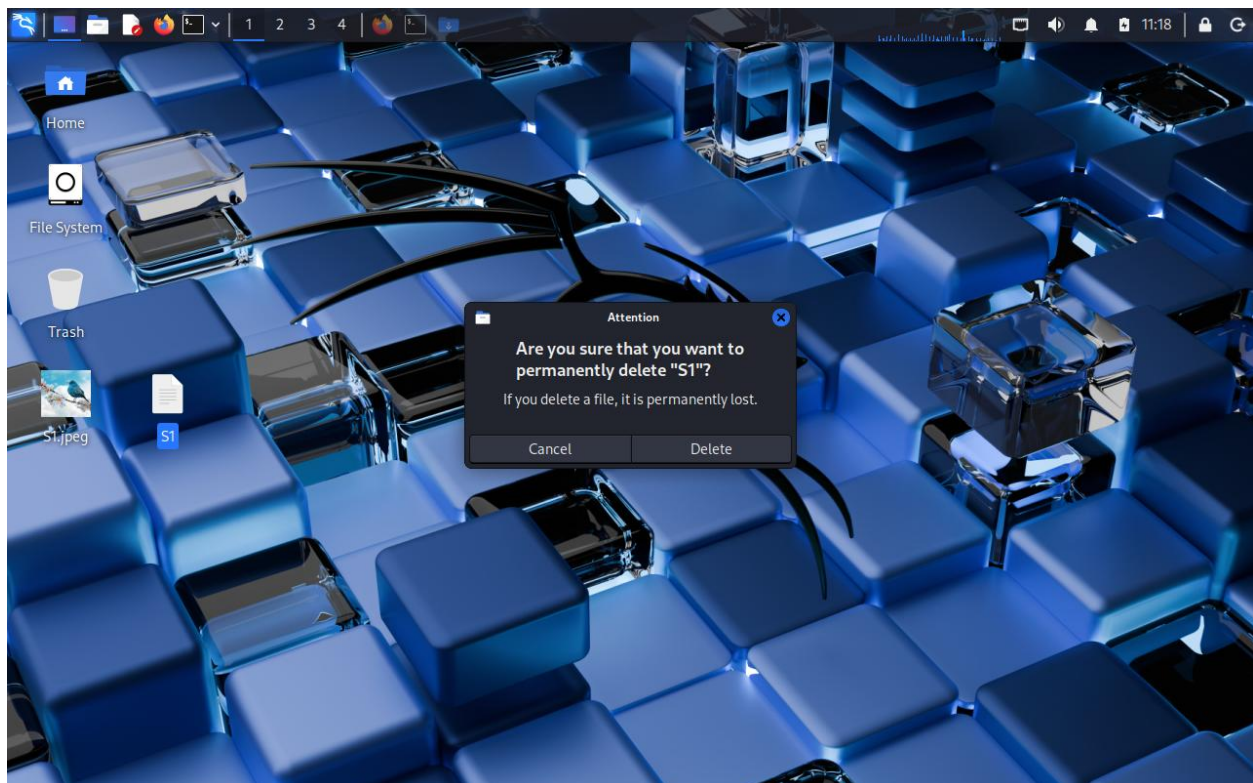
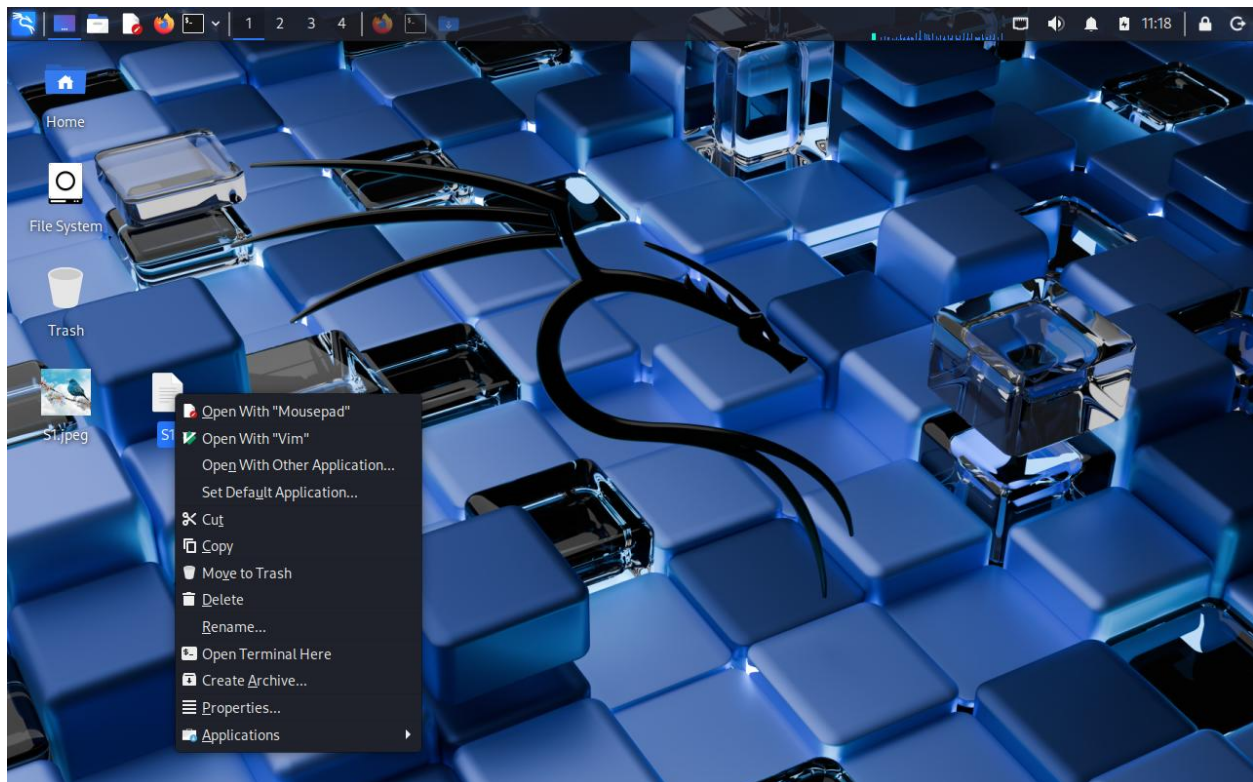
Session Actions Edit View Help
-xf, --extractfile      select file name for extracted data
-xf <filename>         write the extracted data to <filename>
-f, --force             overwrite existing files
-q, --quiet             suppress information messages
-v, --verbose           display detailed information

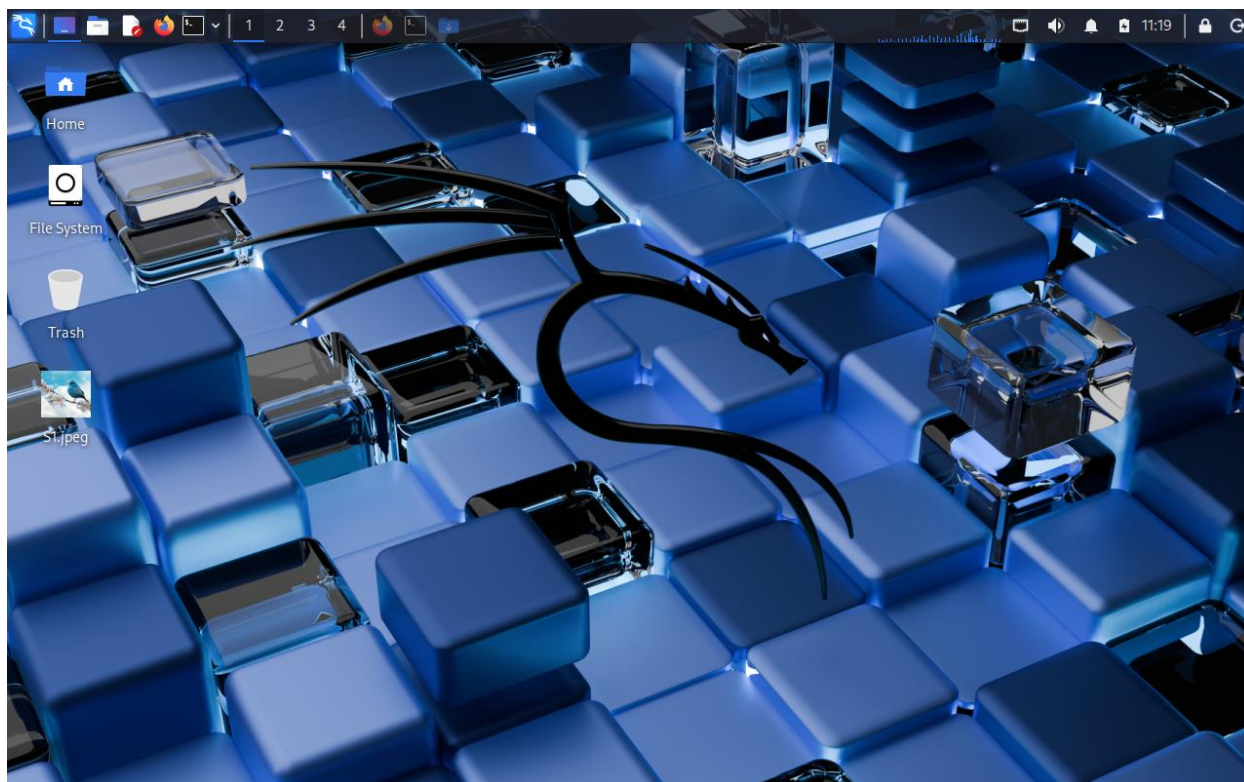
options for the info command:
-p, --passphrase        specify passphrase
-p <passphrase>         use <passphrase> to get info about embedded data

To embed emb.txt in cvr.jpg: steghide embed -cf cvr.jpg -ef emb.txt
To extract embedded data from stg.jpg: steghide extract -sf stg.jpg

(kali@kali)~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
(kali@kali)~$ man steghide
(kali@kali)~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
(kali@kali)~$ cd Desktop
(kali@kali)~/Desktop$ ls
S1 S1.jpeg
(kali@kali)~/Desktop$ cat S1
this is secret message
(kali@kali)~/Desktop$ steghide --embed -ef S1 -cf S1.jpeg
Enter passphrase:
Re-Enter passphrase:
embedding "S1" in "S1.jpeg"... done
(kali@kali)~/Desktop$
```







```
kali@kali: ~/Desktop
Applications
Session Actions Edit View Help
-v, --verbose          display detailed information

options for the info command:
-p, --passphrase       specify passphrase
-p <passphrase>        use <passphrase> to get info about embedded data

To embed emb.txt in cvr.jpg: steghide embed -cf cvr.jpg -ef emb.txt
To extract embedded data from stg.jpg: steghide extract -sf stg.jpg

(kali@kali)~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
(kali@kali)~$ man steghide
(kali@kali)~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
(kali@kali)~$ cd Desktop
(kali@kali)~/Desktop$ ls
S1 S1.jpeg
(kali@kali)~/Desktop$ cat S1
this is secret message
(kali@kali)~/Desktop$ steghide --embed -ef S1 -cf S1.jpeg
Enter passphrase:
Re-Enter passphrase:
embedding "S1" in "S1.jpeg" ... done
(kali@kali)~/Desktop$ ls
S1 S1.jpeg
(kali@kali)~/Desktop$
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ steghide --info S1.jpeg
"S1.jpeg":
  format: jpeg
  capacity: 340.0 Byte
  Try to get information about embedded data ? (y/n) y
Enter passphrase:
  embedded file "S1":
    size: 23.0 Byte
    encrypted: rijndael-128, cbc
    compressed: yes

(kali@kali)-[~/Desktop]
$
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ steghide --info S1.jpeg
"S1.jpeg":
  format: jpeg
  capacity: 340.0 Byte
  Try to get information about embedded data ? (y/n) y
Enter passphrase:
  embedded file "S1":
    size: 23.0 Byte
    encrypted: rijndael-128, cbc
    compressed: yes

(kali@kali)-[~/Desktop]
$ steghide --extract -sf S1.jpeg
Enter passphrase:
wrote extracted data to "S1".

(kali@kali)-[~/Desktop]
$
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ steghide --info S1.jpeg
"S1.jpeg":
  format: jpeg
  capacity: 340.0 Byte
  Try to get information about embedded data ? (y/n) y
  Enter passphrase:
    embedded file "S1":
      size: 23.0 Byte
      encrypted: rijndael-128, cbc
      compressed: yes

(kali@kali)-[~/Desktop]
$ steghide --extract -sf S1.jpeg
Enter passphrase:
wrote extracted data to "S1".

(kali@kali)-[~/Desktop]
$ ls
S1  S1.jpeg

(kali@kali)-[~/Desktop]
$
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)-[~/Desktop]
$ steghide --info S1.jpeg
"S1.jpeg":
  format: jpeg
  capacity: 340.0 Byte
  Try to get information about embedded data ? (y/n) y
  Enter passphrase:
    embedded file "S1":
      size: 23.0 Byte
      encrypted: rijndael-128, cbc
      compressed: yes

(kali@kali)-[~/Desktop]
$ steghide --extract -sf S1.jpeg
Enter passphrase:
wrote extracted data to "S1".

(kali@kali)-[~/Desktop]
$ ls
S1  S1.jpeg

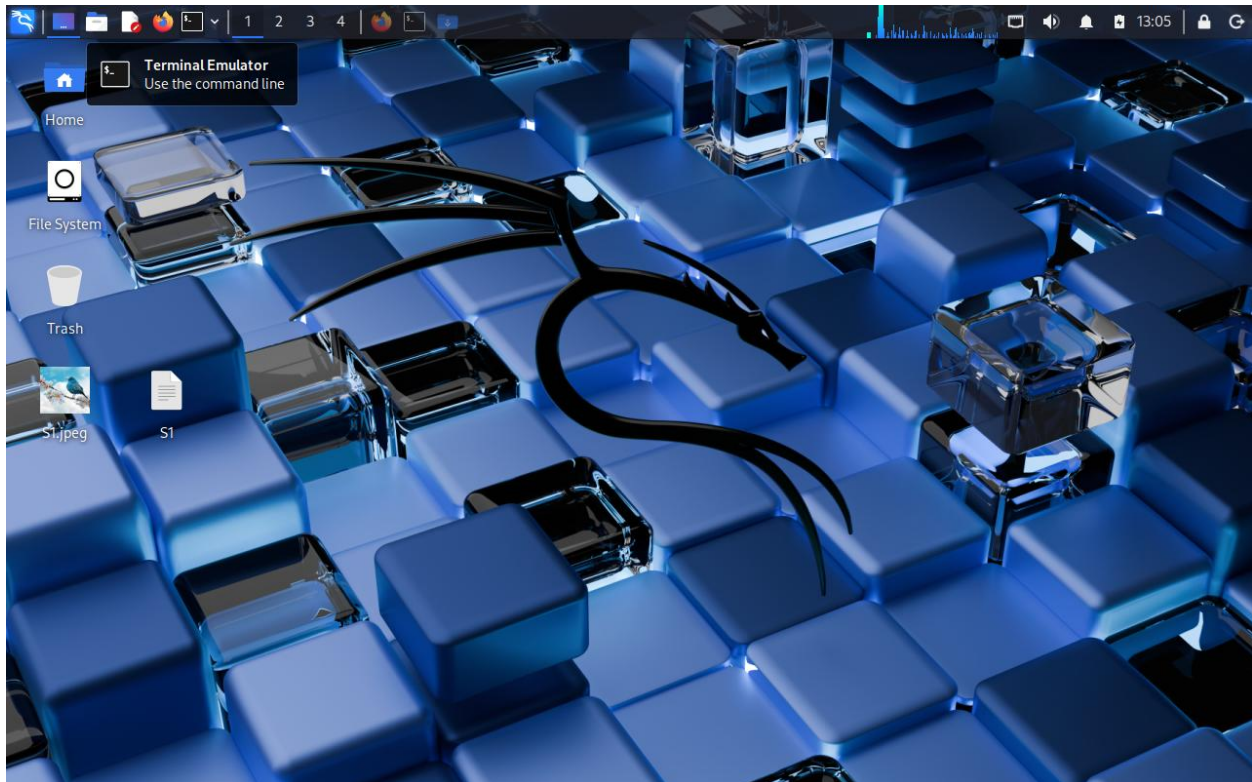
(kali@kali)-[~/Desktop]
$ ca
ca: command not found

(kali@kali)-[~/Desktop]
$ cat s1
cat: s1: No such file or directory

(kali@kali)-[~/Desktop]
$ cat S
cat: S: No such file or directory

(kali@kali)-[~/Desktop]
$ cat S1
this is secret message

(kali@kali)-[~/Desktop]
$
```

Recovering Passwords Techniques:

1. Instant Password Extraction:

The password may occasionally be instantaneously recovered. Only very poorly implemented password protection makes this possible. MS Access is used as the example.

No matter how long a password is, it can always be promptly recovered for an MS Access document.

Time Needed:	Instantly
What is Recovered:	Basic password
Guaranteed outcome	Yes (if this method is applicable)
Passwords that are retrievable:	Older Word and Excel versions, Outlook (short passwords), Access, and Word and Excel in French
Positives	The preferred option is this (if it is possible).

Negatives	extremely unlikely
Supported by the following LastBit software:	Access Password, Word Password & Excel Password

2. Dictionary Attack:

Dictionary Attack makes use of one. Every word in the dictionary will be tested as a password by password crackers. You won't find the word "qwerty" in an ordinary dictionary, but it will undoubtedly be included in a good word list. A good dictionary, also known as a word list, is more than just a dictionary. In fact, many people use this combination of characters as a password. Dictionary Attack typically moves rather quickly. Only if the vocabulary is really vast are noticeable delays likely. The password can only be recovered, though, if it appears in the dictionary. This assumption's likelihood of being accurate is not very great. Though it is advised to try Dictionary Attack first before moving on to the slower Brute Force Attack because it doesn't take long. This approach has a version called Hybrid Dictionary Attack that vastly raises the likelihood of success. The password cracker in this instance looks up every word in the dictionary and all of its synonyms. These might be the same words with various digits added to them, for instance. When the variations include words with two numbers appended to them, the operation is 100 times slower using Hybrid Dictionary Attack than with Dictionary Attack. It takes 10 000 times longer when 4 digits are added.

Time Needed:	many minutes. The amount of time needed may increase significantly in cases where the dictionary is very large or when a hybrid dictionary attack is used, but it is still acceptable.
What is Recovered:	Basic password
Passwords that can be recovered	Any password
Positives	Versatility, minimal time commitment
Negatives	a slim possibility of success

Supported by the following LastBit software:	Word Password, Excel Password, Zip Password, VBA Password, OneNote Password, PowerPoint Password, WinPassword, PwITool
--	--

3. Known Plain Text Attack:

In a known plaintext attack, the attacker is aware of both the ciphertext and the plaintext.

The remaining ciphertext is decrypted using this information. With a selected plaintext attack, the attacker can use the target's key to encrypt a plaintext message of their choosing, giving them access to the ciphertext that results. The encryption key is derived using this information. Attacks of this kind target public key cryptosystems when the attacker is in possession of the public key. You can retrieve password-protected ZIP archives using this technique. It can only be utilized if the archive contains multiple files and the user has access to at least one of them. An executable and numerous DLLs are included in the archive, and among them are standard DLLs that can be discovered unencrypted. As an example, the archive contains multiple Word documents, and the user has a copy of one of them that has been unpacked.

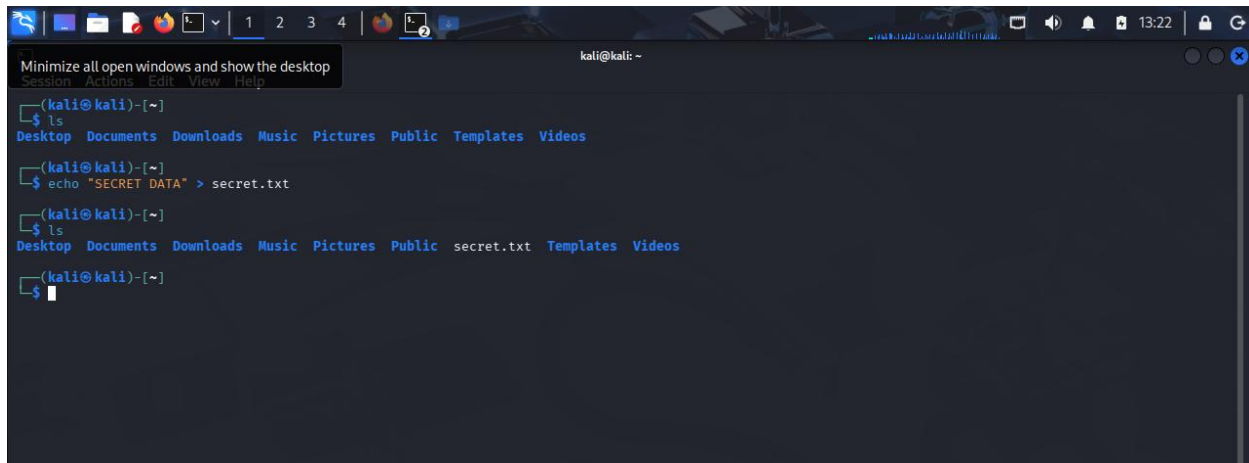
Time Needed:	Numerous hours
What is Recovered:	Unencrypted archive
Requisites/Limitations	One of the files must have isn't encrypted.
Passwords that can be recovered	Zip archive password
Positives	No matter how long the password is, a certain outcome will be obtained in a reasonable amount of time.
Negatives	The password is still a secret.
Supported by the following LastBit software:	Password for Zip

4. Password Variation:

A password issue is frequently the result of faulty typing. The user may enter the password incorrectly or with CAPS LOCK on. Additionally, the user frequently remembers the password's general appearance but not its specifics. In this scenario, we can test every password option by changing the case (password to PASSWORD, Password to PAssword, Password to pASSWORD, etc.), omitting one of the characters, multiplying the characters, inserting the character, replacing the character with the nearby characters, and so on. Typically, there aren't many conceivable combinations, making it possible to test them all quickly.

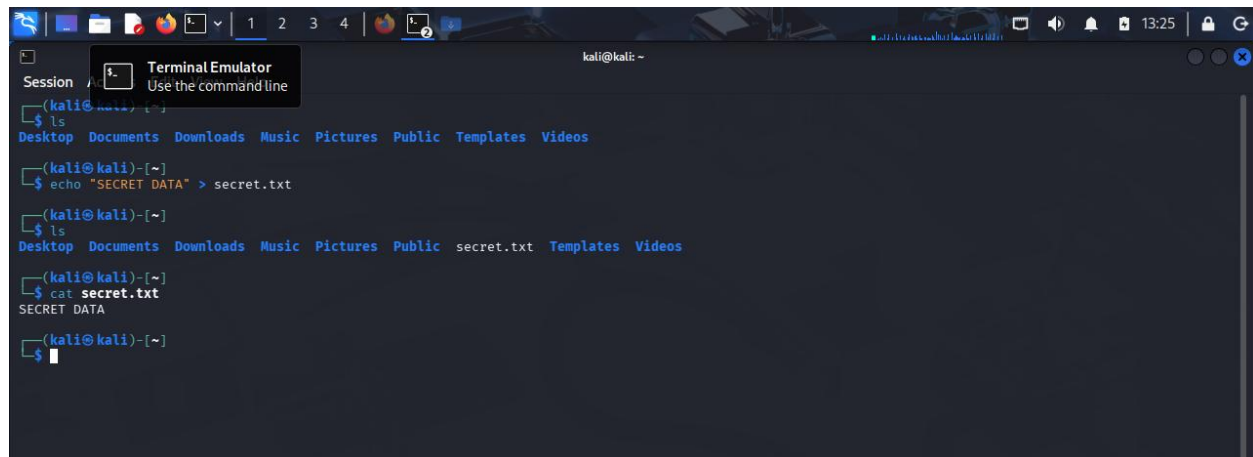
Time Needed:	Depending on the depth of the variance, generally not much
What is Recovered:	Basic password
Requisites/Limitations	The user must be aware of the rough password.
Passwords that can be recovered	Any password
Positives	works really quickly even with long passwords
Negatives	Not usually applicable (only when a rough password is known), and success is not guaranteed.
Supported by the following LastBit software:	a password generator that works with any software that supports dictionary attacks

❖ Implementation:



A screenshot of a Kali Linux terminal window. The window title is "kali@kali: ~". The terminal shows the following commands and output:

```
(kali@kali)-[~]  
$ ls  
Desktop Documents Downloads Music Pictures Public Templates Videos  
(kali@kali)-[~]  
$ echo "SECRET DATA" > secret.txt  
(kali@kali)-[~]  
$ ls  
Desktop Documents Downloads Music Pictures Public secret.txt Templates Videos  
(kali@kali)-[~]  
$
```



A screenshot of a Kali Linux terminal window. The window title is "kali@kali: ~". The terminal shows the following commands and output:

```
(kali@kali)-[~]  
$ ls  
Desktop Documents Downloads Music Pictures Public Templates Videos  
(kali@kali)-[~]  
$ echo "SECRET DATA" > secret.txt  
(kali@kali)-[~]  
$ ls  
Desktop Documents Downloads Music Pictures Public secret.txt Templates Videos  
(kali@kali)-[~]  
$ cat secret.txt  
SECRET DATA  
(kali@kali)-[~]  
$
```



```
kali@kali: ~  
Session Actions Edit View Help  
$ ls  
Desktop Documents Downloads Music Pictures Public Templates Videos  
$ echo "SECRET DATA" > secret.txt  
$ ls  
Desktop Documents Downloads Music Pictures Public secret.txt Templates Videos  
$ cat secret.txt  
SECRET DATA  
$ zip --encrypt protected.zip secret.txt  
Enter password:  
Verify password:  
adding: secret.txt (stored 0%)  
$
```

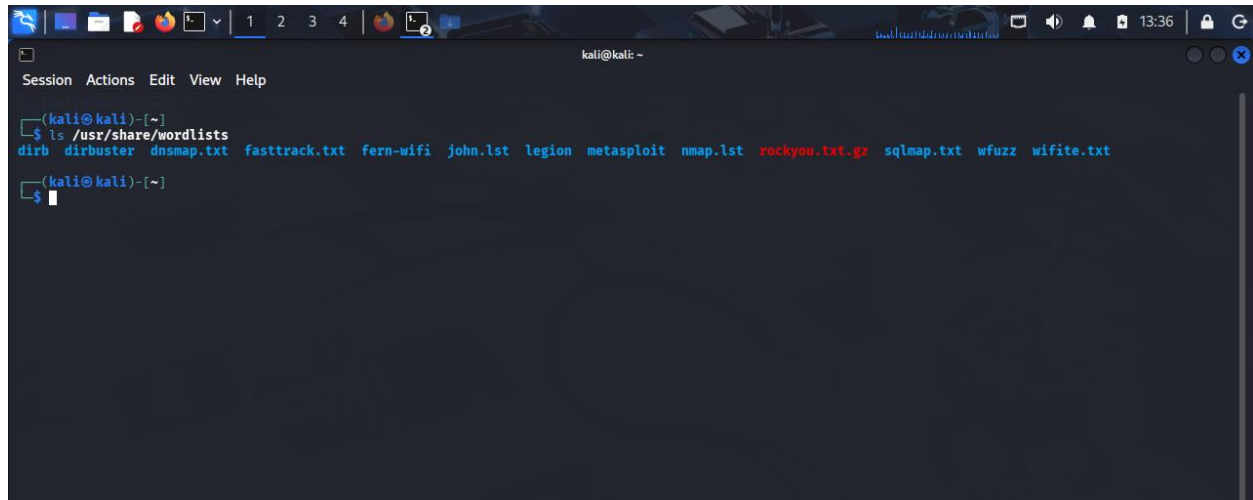
```
kali@kali: ~  
Session Actions Edit View Help  
$ ls  
Desktop Documents Downloads Music Pictures Public Templates Videos  
$ echo "SECRET DATA" > secret.txt  
$ ls  
Desktop Documents Downloads Music Pictures Public secret.txt Templates Videos  
$ cat secret.txt  
SECRET DATA  
$ zip --encrypt protected.zip secret.txt  
Enter password:  
Verify password:  
adding: secret.txt (stored 0%)  
$ unzip protected.zip  
Archive: protected.zip  
[protected.zip] secret.txt password:  
password incorrect--reenter:  
password incorrect--reenter:  
skipping: secret.txt incorrect password  
$
```

• Hash Extraction:

```
kali@kali: ~  
Session Actions Edit View Help  
$ ls  
Desktop Documents Downloads Music Pictures Public Templates Videos  
$ echo "SECRET DATA" > secret.txt  
$ ls  
Desktop Documents Downloads Music Pictures Public secret.txt Templates Videos  
$ cat secret.txt  
SECRET DATA  
$ zip --encrypt protected.zip secret.txt  
Enter password:  
Verify password:  
adding: secret.txt (stored 0%)  
$ unzip protected.zip  
Archive: protected.zip  
[protected.zip] secret.txt password:  
password incorrect--reenter:  
password incorrect--reenter:  
skipping: secret.txt          incorrect password  
$ zip2john protected.zip > zip_hash.txt  
Created directory: /home/kali/.john  
ver 1.0 efh 5455 efh 7875 protected.zip/secret.txt PKZIP Encr: 2b chk, TS_chk, cmplen=24, decmplen=12, crc=E394FA91 ts=6AB8 cs=6ab8 type=0  
$
```

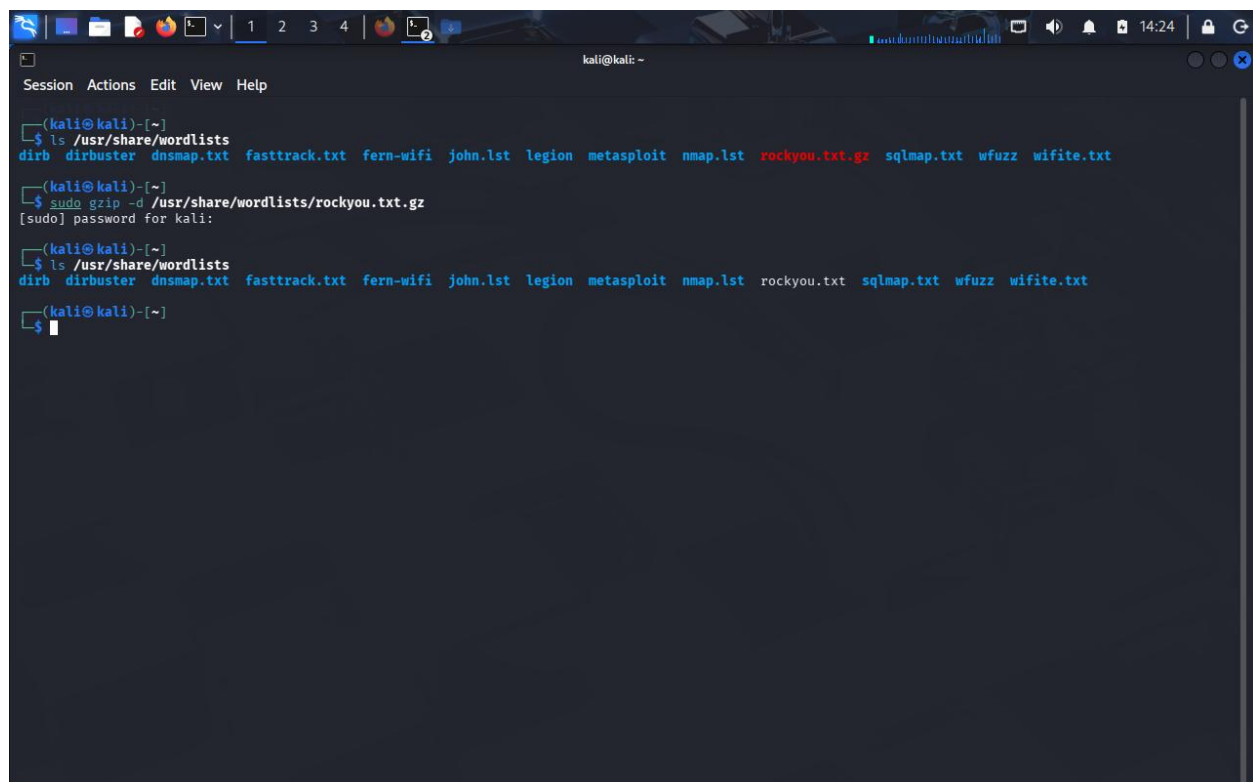
```
kali@kali: ~  
Applications  
Session Actions Edit View Help  
$ ls  
Desktop Documents Downloads Music Pictures protected.zip Public secret.txt Templates Videos zip_hash.txt  
$ cat zip_hash.txt  
protected.zip/secret.txt:$pkzip$1*2*0*18*c*e394fa91*0*44*0*18*6ab8*b1ea0ccee41e1b43dcd2b5264bd2006564b62e5d1e59495e*$/pkzip$:secret.txt:protected.zip::pr  
otected.zip  
$
```

- Dictionary Attacks:



A terminal window on a Kali Linux system. The prompt is `(kali@kali)-[~]`. The user enters `$ ls /usr/share/wordlists`, which outputs a list of files: `dirb dirbuster dnsmap.txt fasttrack.txt fern-wifi john.lst legion metasploit nmap.lst rockyou.txt.gz sqlmap.txt wfuzz wifite.txt`. The prompt returns to `(kali@kali)-[~]` with a dollar sign.

```
(kali@kali)-[~]  
$ ls /usr/share/wordlists  
dirb dirbuster dnsmap.txt fasttrack.txt fern-wifi john.lst legion metasploit nmap.lst rockyou.txt.gz sqlmap.txt wfuzz wifite.txt  
(kali@kali)-[~]  
$
```



A terminal window on a Kali Linux system. The prompt is `(kali@kali)-[~]`. The user enters `$ ls /usr/share/wordlists`, showing the same list of files as the previous screenshot. Then, the user enters `$ sudo gzip -d /usr/share/wordlists/rockyou.txt.gz`. A prompt `[sudo] password for kali:` appears. After an invisible password entry, the prompt returns to `(kali@kali)-[~]`. The user enters `$ ls /usr/share/wordlists` again, and the output now includes `rockyou.txt` instead of `rockyou.txt.gz`. The prompt returns to `(kali@kali)-[~]` with a dollar sign.

```
(kali@kali)-[~]  
$ ls /usr/share/wordlists  
dirb dirbuster dnsmap.txt fasttrack.txt fern-wifi john.lst legion metasploit nmap.lst rockyou.txt.gz sqlmap.txt wfuzz wifite.txt  
(kali@kali)-[~]  
$ sudo gzip -d /usr/share/wordlists/rockyou.txt.gz  
[sudo] password for kali:  
(kali@kali)-[~]  
$ ls /usr/share/wordlists  
dirb dirbuster dnsmap.txt fasttrack.txt fern-wifi john.lst legion metasploit nmap.lst rockyou.txt sqlmap.txt wfuzz wifite.txt  
(kali@kali)-[~]  
$
```

```
kali@kali: ~  
Applications  
Session Actions Edit View Help  
[kali@kali]~  
$ ls /usr/share/wordlists  
dirb dirbuster dnsmap.txt fasttrack.txt fern-wifi john.lst legion metasploit nmap.lst rockyou.txt.gz sqlmap.txt wfuzz wifite.txt  
[kali@kali]~  
$ sudo gzip -d /usr/share/wordlists/rockyou.txt.gz  
[sudo] password for kali:  
[kali@kali]~  
$ ls /usr/share/wordlists  
dirb dirbuster dnsmap.txt fasttrack.txt fern-wifi john.lst legion metasploit nmap.lst rockyou.txt sqlmap.txt wfuzz wifite.txt  
[kali@kali]~  
$ john zip_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (PKZIP [32/64])  
fopen: /usr/share/wordlists/rockyou.txt: No such file or directory  
[kali@kali]~  
$ john zip_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (PKZIP [32/64])  
Press 'q' or Ctrl-C to abort, almost any other key for status  
asd123 (protected.zip/secret.txt)  
1g 0:00:00:00 DONE (2026-06-18 14:28) 11.11g/s 22044p/s 22044c/s 22044C/s amore..peaches1  
Use the "--show" option to display all of the cracked passwords reliably  
Session completed.  
[kali@kali]~  
$
```

```
kali@kali: ~  
Applications  
Session Actions Edit View Help  
[kali@kali]~  
$ ls /usr/share/wordlists  
dirb dirbuster dnsmap.txt fasttrack.txt fern-wifi john.lst legion metasploit nmap.lst rockyou.txt.gz sqlmap.txt wfuzz wifite.txt  
[kali@kali]~  
$ sudo gzip -d /usr/share/wordlists/rockyou.txt.gz  
[sudo] password for kali:  
[kali@kali]~  
$ ls /usr/share/wordlists  
dirb dirbuster dnsmap.txt fasttrack.txt fern-wifi john.lst legion metasploit nmap.lst rockyou.txt sqlmap.txt wfuzz wifite.txt  
[kali@kali]~  
$ john zip_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (PKZIP [32/64])  
fopen: /usr/share/wordlists/rockyou.txt: No such file or directory  
[kali@kali]~  
$ john zip_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (PKZIP [32/64])  
Press 'q' or Ctrl-C to abort, almost any other key for status  
asd123 (protected.zip/secret.txt)  
1g 0:00:00:00 DONE (2026-06-18 14:28) 11.11g/s 22044p/s 22044c/s 22044C/s amore..peaches1  
Use the "--show" option to display all of the cracked passwords reliably  
Session completed.  
[kali@kali]~  
$ john --show zip_hash.txt  
protected.zip/secret.txt:asd123:secret.txt:protected.zip::protected.zip  
1 password hash cracked, 0 left  
[kali@kali]~  
$
```

```
kali@kali: ~  
Session Actions Edit View Help  
kali@kali ~  
$ ls /usr/share/wordlists  
dirb dirbuster dnsmap.txt fasttrack.txt fern-wifi john.lst legion metasploit nmap.lst rockyou.txt.gz sqlmap.txt wfuzz wifite.txt  
kali@kali ~  
$ sudo gzip -d /usr/share/wordlists/rockyou.txt.gz  
[sudo] password for kali:  
kali@kali ~  
$ ls /usr/share/wordlists  
dirb dirbuster dnsmap.txt fasttrack.txt fern-wifi john.lst legion metasploit nmap.lst rockyou.txt sqlmap.txt wfuzz wifite.txt  
kali@kali ~  
$ john zip_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (PKZIP [32/64])  
fopen: /usr/share/wordlists/rockyou.txt: No such file or directory  
kali@kali ~  
$ john zip_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (PKZIP [32/64])  
Press 'q' or Ctrl-C to abort, almost any other key for status  
asd123 (protected.zip/secret.txt)  
1g 0:00:00:00 DONE (2026-06-18 14:28) 11.11g/s 22044p/s 22044c/s 22044C/s amore..peaches1  
Use the "--show" option to display all of the cracked passwords reliably  
Session completed.  
kali@kali ~  
$ john --show zip_hash.txt  
protected.zip/secret.txt:asd123:secret.txt::protected.zip  
1 password hash cracked, 0 left  
kali@kali ~  
$ ls  
Desktop Documents Downloads Music Pictures protected.zip Public secret.txt Templates Videos zip_hash.txt  
kali@kali ~  
$ rm protected.zip zip_hash.txt  
kali@kali ~  
$
```

Complex password

```
kali@kali: ~  
Session Actions Edit View Help  
kali@kali ~  
$ ls  
Desktop Documents Downloads Music Pictures Public secret.txt Templates Videos  
kali@kali ~  
$ zip --encrypt protected.zip secret.txt  
Enter password:  
Verify password:  
adding: secret.txt (stored 0%)  
kali@kali ~  
$
```



```
kali@kali: ~  
$ ls  
Desktop  Documents  Downloads  Music  Pictures  Public  secret.txt  Templates  Videos  
$ zip --encrypt protected.zip secret.txt  
Enter password:  
Verify password:  
adding: secret.txt (stored 0%)  
$ zip2john protected.zip > zip_hash.txt  
ver 1.0 efh 5455 efh 7875 protected.zip/secret.txt PKZIP Encr: 2b chk, TS_chk, cmplen=24, decmlen=12, crc=E394FA91 ts=6AB8 cs=6ab8 type=0  
$ john zip_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt  
Using default input encoding: UTF-8  
Loaded 1 password hash (PKZIP [32/64])  
Press 'q' or Ctrl-C to abort, almost any other key for status  
0g 0:00:00:02 DONE (2026-06-18 14:39) 0g/s 6579Kp/s 6579Kc/s 6579Kc/s 0123456789 ..*7;Vamos!  
Session completed.  
$
```

• Brute-Force Methods

```
kali@kali: ~  
$ john zip_hash.txt --incremental=LowerNum --min-length=4 --max-length=5  
Using default input encoding: UTF-8  
Loaded 1 password hash (PKZIP [32/64])  
Press 'q' or Ctrl-C to abort, almost any other key for status  
x2f3h (protected.zip/secret.txt)  
1g 0:00:00:04 DONE (2026-06-18 14:44) 0.2016g/s 9642Kp/s 9642Kc/s 9642Kc/s x2fbd..x2fx5  
Use the "--show" option to display all of the cracked passwords reliably  
Session completed.  
$ john --show zip_hash.txt  
protected.zip/secret.txt:x2f3h:secret.txt:protected.zip::protected.zip  
1 password hash cracked, 0 left  
$
```

Reference:

- [91.pdf \(acadpubl.eu\)](#)
- [Stegnography20191120-9982-1h76qc4-with-cover-page-v2.pdf \(d1wqtxts1xzle7.cloudfront.net\)](#)
- [What Is a Host Protected Area? \(with pictures\) \(easytechjunkie.com\)](#)
- [Hiding data with Host Protected Area \(HPA\) in Linux - Checkmate \(niiconsulting.com\)](#)
- [Two Data-Hiding Techniques \(itprotoday.com\)](#)
- [Plaintext Attack - an overview | ScienceDirect Topics](#)
- [Password Recovery Methods \(lastbit.com\)](#)